

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología



Universidad
Gerardo Barrios

Informe Final de Proyecto
Trabajo de Graduación

Modalidad Preespecialización
en la Especialidad de:

Desarrollo de Aplicaciones con Inteligencia
Artificial.

“Gerardito: IA para recomendación de carreras
universitarias.”

Presentado por:

Ayala Santos Fatima del Carmen – SMIS003321

Chevez Sánchez Fernando Ruben – SMIS890919

Reyes Medina Meylin Nohely – SMIS032721

San Miguel, 23 de agosto de 2026

Índice

1. RESUMEN DEL PROYECTO	3
2. MARCO CONCEPTUAL.....	5
2.1 Inteligencia Artificial	5
2.2 Inteligencia Artificial Generativa	5
2.3 Large Language Model (LLM)	6
2.4 Procesamiento de Lenguaje Natural	6
2.5 Sistemas de recomendación.....	7
2.6 Orientación Vocacional	8
2.7 Chatbots y Sistemas Conversacionales	8
2.8 Ingeniería de prompts.....	9
2.9 Seguridad de aplicaciones basadas en LLM	9
2.10 APIs REST y FastAPI	10
2.11 React, TypeScript y Vite	10
2.12 PostgreSQL y persistencia de datos	10
2.13 SQLAlchemy y acceso asíncrono.....	11
2.14 Integración continua y pruebas automatizadas.....	11
2.15 Despliegue Continuo (Continuous Deployment)	12
2.16 Docker y Docker Compose.....	12
2.17 Observabilidad y métricas.....	13
2.18 Arquitectura desacoplada y separación por capas	13
2.19 Ética y uso responsable en orientación vocacional	14
2.20 Síntesis conceptual aplicada al proyecto.....	14
3. ANALISIS DEL PROBLEMA.....	15
3.1 Contexto de la situación problemática.....	15
3.2 Formulación del problema.....	15
3.3 Causas identificadas.....	15
3.4 Consecuencias potenciales	16
3.5 Usuarios y beneficiarios	16
3.6 Requerimientos funcionales identificados.....	16
4. PROPUESTA DE SOLUCION	18
4.1 Descripción general	18

4.2 Objetivo general	18
4.3 Objetivos específicos	18
4.3.1 Arquitectura de comunicación y API	18
4.4 Arquitectura de la solución	19
4.5 Flujo funcional de la aplicación	20
4.6 Componentes tecnológicos	20
4.7 Datos utilizados	21
4.8 Infraestructura y despliegue	22
4.9 Justificación de la solución	22
5. METODOLOGÍA DE DESARROLLO DEL PROYECTO	23
5.1 Etapa 1. Análisis y definición	23
5.2 Etapa 2. Diseño de arquitectura	23
5.3 Etapa 3. Desarrollo del backend	23
5.4 Etapa 4. Desarrollo del frontend	23
5.5 Etapa 5. Integración del modelo de IA	24
5.6 Etapa 6. Procesamiento NLP	24
5.7 Etapa 7. Persistencia	24
5.8 Etapa 8. Contenerización	25
5.9 Etapa 9. Seguridad	25
5.10 Etapa 10. Pruebas y CI/CD	25
5.11 Etapa 11. Observabilidad	26
5.12 Etapa 12. Despliegue	26
5.13 Etapa 13. Recuperación y rollback	27
5.14 Etapa 14. Verificación del despliegue	27
5.15 Integración final de las seis sesiones	27
5.16 Contratos, salud, versión y trazabilidad	28
5.17 Release, pruebas de humo y rollback	29
6. RESULTADOS ALCANZADOS	29
6.1 Cumplimiento funcional	29
6.2 Resultado de la interfaz de usuario	30
6.3 Resultado del motor de recomendación	31
6.4 Resultado del filtro semántico	32

6.5 Resultado de persistencia	32
6.6 Resultado de pruebas y calidad.....	33
6.7 Resultado de observabilidad	34
6.8 Resultado de despliegue	34
6.9 Evidencias	34
6.10 Robustez operacional.....	38
6.11 Evidencia de rendimiento y observabilidad	38
7. CONCLUSIONES GENERALES	40
7.1 Cumplimiento del objetivo del proyecto	40
7.2 Aporte tecnológico	40
7.3 Aporte a la comunidad educativa	40
7.4 Nuevos conocimientos adquiridos.....	40
7.5 Limitaciones actuales	41
7.6 Escalabilidad y trabajo futuro.....	41
7.7 Riesgo prioritario, límites y continuidad.....	42
8. REFERENCIAS ESTUDIADAS	42

1. RESUMEN DEL PROYECTO

El presente informe documenta el desarrollo y estado final de “Gerardito”, una aplicación web de orientación vocacional inteligente orientada a apoyar a los aspirantes de la Universidad Gerardo Barrios (UGB) en la exploración de carreras universitarias. Tradicionalmente, las universidades han implementado test vocacionales sustentados en sistemas basados en reglas, contruidos a partir del conocimiento experto de profesionales en psicología y educación esto puede presentar limitaciones al basarse en formularios rígidos que no permiten interpretar con flexibilidad la forma natural de comunicación, incluyendo variaciones lingüísticas, errores ortográficos y formas de comunicación propia en que los estudiantes expresan sus intereses, habilidades y expectativas.

La solución desarrollada permite que el aspirante seleccione y escriba sus habilidades e intereses. Estos datos son recibidos por una interfaz web construida con React, TypeScript y Vite y enviados a una API REST implementada con FastAPI. Antes de llegar al componente de inteligencia artificial, la información se somete a validación mediante esquemas Pydantic y a un filtro semántico de seguridad. Posteriormente, el sistema utiliza un modelo Gemma 4 31B mediante Ollama Cloud, orquestado por la lógica del backend, para realizar el diagnóstico y cruzar el perfil del usuario con el catálogo académico de la UGB. El resultado principal consiste en tres carreras sugeridas acompañadas de un porcentaje de compatibilidad y una justificación comprensible.

La aplicación incorpora además una fase de exploración controlada de las carreras sugeridas y una etapa final de reseña. Las reseñas son procesadas mediante un enfoque híbrido que combina el modelo de lenguaje con spaCy para apoyar el análisis de sentimiento y la extracción de información lingüística. En cuanto a persistencia, se utiliza PostgreSQL mediante SQLAlchemy en modalidad asíncrona, con estructuras para almacenar sesiones, diagnósticos, exploraciones y reseñas. La arquitectura opera de forma desacoplada y se encuentra contenerizada mediante Docker Compose, organizando los servicios de base de datos, API y frontend.

El proyecto incorpora validación de datos, seguridad de entradas, orquestación de modelos, procesamiento de lenguaje natural, pruebas automatizadas, integración continua y observabilidad. GitHub Actions ejecuta la suite de pruebas ante cambios en el repositorio, mientras que el backend dispone de mecanismos para registrar tiempos de respuesta y métricas acumuladas mediante el endpoint `/metrics`. El README actualizado del proyecto reporta ocho endpoints documentados bajo el prefijo `/api/v1/`, documentación Swagger automática y una arquitectura de cinco capas completamente implementada.

Como infraestructura de despliegue, se ha incorporado una máquina virtual en Oracle y un dominio gratuito gestionado mediante DigitalPlat FreeDomain. El documento técnico disponible confirma que la aplicación está preparada para ejecutarse mediante Docker Compose y que la arquitectura ya se encuentra organizada para separar interfaz, API, inteligencia artificial, persistencia y operación. En esta etapa, el uso de Ollama Cloud elimina la dependencia de una GPU local para la inferencia del modelo, aunque introduce una dependencia externa de conectividad.

El resultado final es un prototipo funcional que materializa una solución de orientación vocacional con interacción moderna y una arquitectura reproducible. En este proyecto presentamos una evolución respecto del anteproyecto: la propuesta inicial consideraba un sistema híbrido basado en reglas y algoritmos clásicos de Machine Learning, pero durante el proceso de desarrollo se priorizó una arquitectura conversacional y semántica basada en IA generativa, manteniendo como eje el catálogo institucional y la validación del perfil del aspirante. Esta evolución responde al alcance técnico efectivamente implementado y permite documentar el producto que realmente se encuentra operativo al cierre del Proyecto.

2. MARCO CONCEPTUAL

2.1 Inteligencia Artificial

La inteligencia artificial (IA) comprende un conjunto amplio de métodos computacionales orientados a resolver problemas que requieren capacidades asociadas tradicionalmente con la inteligencia humana, como percepción, razonamiento, clasificación, predicción, aprendizaje y generación de contenido. En el contexto educativo, la IA puede utilizarse como apoyo para analizar información de los estudiantes, automatizar procesos y construir herramientas interactivas que faciliten la toma de decisiones.

El proyecto Gerardito se ubica dentro de esta perspectiva aplicada de que la IA puede utilizarse como apoyo para analizar información de los estudiantes, automatizar procesos y construir herramientas interactivas que faciliten la toma de decisiones no pretende sustituir el criterio profesional de orientación, sino ofrecer un recurso tecnológico para organizar información del aspirante y producir una recomendación explicable y contextualizada.

La IA no se limita a una clasificación cerrada, el sistema recibe datos categóricos y texto libre y utiliza un modelo de lenguaje para interpretar semánticamente la información. Esto permite trabajar con una entrada más cercana a la forma en que el estudiante se expresa en una conversación. La utilidad se encuentra en la capacidad de transformar lenguaje no estructurado en elementos que puedan ser relacionados con la oferta académica de la UGB.

2.2 Inteligencia Artificial Generativa

La IA generativa se caracteriza por producir nuevas salidas a partir de patrones aprendidos por los modelos. A diferencia de un sistema puramente basado en reglas, un modelo generativo puede producir explicaciones, reformular información y responder en lenguaje natural. Esta propiedad resulta pertinente en orientación vocacional porque la recomendación no solo debe seleccionar una opción, sino también explicar por qué una determinada carrera presenta afinidad con el perfil ingresado.

Una salida lingüísticamente coherente no garantiza por sí misma exactitud ni pertinencia. En Gerardito se aborda este problema mediante instrucciones de dominio, validación Pydantic, un filtro semántico y un catálogo institucional que se incorpora al razonamiento de la aplicación. Así, la generación se utiliza dentro de un flujo predefinido en lugar de permitir una conversación sin restricciones. Este diseño es consistente con el objetivo del sistema: generar recomendaciones dentro de la oferta de la universidad y no responder como un asistente de propósito general.

2.3 Large Language Model (LLM)

Un Large Language Model (LLM) es un modelo entrenado para procesar y generar lenguaje a gran escala. Los LLM modernos utilizan arquitecturas basadas en transformadores y aprenden relaciones estadísticas entre unidades lingüísticas a partir de grandes colecciones de datos. La capacidad de contextualización permite que una misma idea pueda interpretarse con diferentes formulaciones y que el modelo produzca respuestas coherentes con una instrucción determinada.

El modelo de LLM seleccionado es Gemma 4 31B. La documentación oficial de Google identifica a Gemma 4 como una familia de modelos abiertos con variantes de diferentes tamaños, incluyendo una versión densa de 31B parámetros, orientada a tareas de generación de texto, razonamiento y otros flujos de trabajo. La solución utiliza esta capacidad a través de Ollama Cloud, lo que permite acceder al modelo sin exigir una GPU local de alto rendimiento en la máquina donde se ejecuta la aplicación.

La integración mediante Ollama Cloud se realiza a través de la API del proveedor. La documentación oficial señala que los modelos cloud pueden ejecutarse mediante el servicio remoto y que el acceso programático requiere autenticación mediante una API key. Esta característica explica una de las limitaciones actuales del sistema: las funciones de diagnóstico, exploración y reseña dependen de la conectividad con el servicio de inferencia.

2.4 Procesamiento de Lenguaje Natural

El procesamiento de lenguaje natural (NLP) agrupa técnicas para analizar, representar e interpretar lenguaje humano mediante computadoras. Entre sus tareas se

encuentran la tokenización, lematización, identificación de categorías gramaticales, extracción de entidades, análisis de sentimiento y otras formas de análisis lingüístico. En un sistema de orientación, NLP permite pasar de frases del usuario a representaciones que puedan utilizarse en etapas posteriores del razonamiento.

Para el procesamiento de Lenguaje Natural Gerardito utiliza spaCy con el modelo en español es_core_news_sm. Su función se relaciona principalmente con el tratamiento lingüístico de las reseñas finales y con el apoyo a la extracción sintáctica. El diseño combina el procesamiento del LLM con herramientas NLP tradicionales: el modelo generativo maneja el razonamiento semántico y la generación de lenguaje, mientras spaCy proporciona operaciones lingüísticas determinísticas o estructuradas. Este enfoque híbrido permite separar tareas y evita cargar todo el procesamiento en una sola tecnología.

2.5 Sistemas de recomendación

Un sistema de recomendación es una aplicación que analiza información sobre usuarios, elementos y contexto para proponer alternativas que podrían ser relevantes. En general, los recomendadores pueden apoyarse en reglas, filtrado colaborativo, contenido, modelos de aprendizaje automático o combinaciones híbridas. En orientación vocacional, el elemento recomendado es la carrera universitaria y el perfil del usuario está compuesto por intereses, habilidades y características expresadas en la interacción.

La solución implementada adopta una estrategia basada en afinidad semántica y conocimiento institucional. El sistema no intenta aprender de manera autónoma una “verdad” sobre el usuario, sino interpretar la información recibida y compararla con el catálogo de la UGB. Es por eso que el resultado se expresa como un conjunto reducido de opciones acompañadas por una justificación. Esto permite comprender la recomendación y utilizarla como apoyo para la decisión, en lugar de presentarla como una predicción absoluta.

2.6 Orientación Vocacional

La orientación vocacional busca apoyar a las personas en la identificación de intereses, habilidades, preferencias y posibilidades de formación que contribuyan a decisiones académicas y profesionales informadas. En educación superior, este proceso adquiere relevancia porque la selección de una carrera se relaciona con expectativas, afinidad personal, desempeño y permanencia. El anteproyecto del equipo planteó precisamente la necesidad de fortalecer un proceso de orientación que, en su forma tradicional, puede estar condicionado por reglas estáticas y formularios rígidos

En la Orientación Vocacional Gerardito se inserta como un mecanismo de apoyo. La aplicación busca que el aspirante pueda expresarse con menos fricción y reciba una explicación comprensible de las opciones sugeridas. La solución no debe interpretarse como sustituto de la orientación profesional, sino como un primer recurso de exploración que facilite al estudiante conocer alternativas de la oferta académica institucional.

2.7 Chatbots y Sistemas Conversacionales

Un chatbot es un sistema que mantiene una interacción con el usuario mediante lenguaje natural. Los chatbots tradicionales pueden basarse en reglas o intenciones predefinidas, mientras que los sistemas actuales pueden incorporar modelos generativos y recuperación de información. La ventaja de una interfaz conversacional es que permite continuar una exploración sin obligar al usuario a navegar únicamente por formularios o menús.

En la versión final de Gerardito, la conversación está controlada por fases. Primero se capturan habilidades e intereses; luego se genera un diagnóstico; después se ofrece la posibilidad de explorar una carrera sugerida y finalmente se registra una reseña. Este flujo evita que la aplicación se comporte como un chatbot de propósito general y concentra la conversación en el dominio vocacional. La interfaz React funciona como capa de presentación y consume la API de manera desacoplada.

2.8 Ingeniería de prompts

La ingeniería de prompts consiste en diseñar las instrucciones que orientan el comportamiento de un modelo de lenguaje. En aplicaciones de IA generativa, el prompt cumple una función de especificación: delimita el rol del modelo, la información de entrada, el formato de salida y las restricciones que debe respetar. Cuanto más sensible sea la tarea, mayor importancia adquiere definir controles y criterios verificables.

En Gerardito, la ingeniería de prompts se utiliza para mantener al modelo dentro del dominio de orientación vocacional y para estructurar las salidas. El catálogo académico se incorpora al contexto de la solicitud, mientras la aplicación valida previamente los datos. Esto ayuda a reducir respuestas fuera de contexto y facilita que la interfaz interprete el resultado. No obstante, un prompt no debe considerarse una barrera de seguridad absoluta; requiere complementarse con validación de entrada, controles de aplicación y pruebas.

2.9 Seguridad de aplicaciones basadas en LLM

Las aplicaciones que integran modelos de lenguaje presentan riesgos particulares porque el usuario puede proporcionar instrucciones que interfieran con el comportamiento esperado del modelo. OWASP identifica la prompt injection como una categoría relevante de riesgo en aplicaciones LLM, ya que una entrada manipulada puede alterar las respuestas del sistema o intentar hacer que el modelo ignore instrucciones previas.

En Gerardito, la seguridad se aborda mediante varias capas. Pydantic valida la estructura y los tipos de datos; la aplicación limita las condiciones de entrada; el filtro semántico clasifica solicitudes como válidas o inválidas; y las instrucciones del servicio de IA restringen el dominio de las respuestas. Este esquema responde a un principio de defensa por capas: ninguna medida aislada se considera suficiente. Además, el hecho de que la inferencia se realice mediante Ollama Cloud implica que las credenciales deben gestionarse mediante variables de entorno y que la conectividad externa constituye parte del modelo de amenaza operativo.

2.10 APIs REST y FastAPI

Una API REST organiza la comunicación entre componentes mediante recursos y operaciones HTTP. En una arquitectura desacoplada, la API permite que el frontend evolucione de manera independiente del backend y que diferentes clientes consuman las mismas capacidades de negocio. FastAPI es un framework de Python orientado a la construcción de APIs y utiliza tipos de Python y Pydantic para validación, además de generar documentación interactiva basada en OpenAPI.

FastAPI constituye la capa central de integración. La interfaz React no contiene la lógica del diagnóstico ni se conecta directamente con la base de datos o el servicio de IA. En su lugar, envía solicitudes a los endpoints del backend, que valida, procesa y devuelve resultados. Esta separación favorece la mantenibilidad, las pruebas y una eventual sustitución de la interfaz sin rehacer la lógica principal.

2.11 React, TypeScript y Vite

React es una biblioteca para construir interfaces mediante componentes. Su enfoque permite dividir la presentación en unidades reutilizables y administrar el estado de la interacción. TypeScript agrega tipado estático al código de JavaScript y ayuda a describir estructuras de datos, props y estados con mayor precisión. La documentación oficial de React presenta la integración con TypeScript como una vía para proporcionar tipos a los componentes y mejorar la verificación del código.

El proyecto utiliza React + TypeScript + Vite para construir una SPA (Single Page Application). Esta elección responde a la necesidad de representar un proceso de varias fases sin recargar continuamente la página.

2.12 PostgreSQL y persistencia de datos

PostgreSQL es un sistema de gestión de bases de datos relacional que ofrece estructuras para almacenar datos de manera consistente y relacionarlos mediante claves. En una aplicación web, la base de datos permite conservar información entre sesiones del proceso y facilitar consultas posteriores. La documentación oficial de PostgreSQL cubre el lenguaje SQL, administración, programación e interfaces para clientes.

PostgreSQL resuelve una limitación presente en etapas anteriores del proyecto: la pérdida del historial cuando el proceso dependía exclusivamente de memoria de sesión. La persistencia permite mantener trazabilidad de la interacción y deja preparada la aplicación para análisis posteriores, aunque el sistema todavía no incorpora autenticación de usuarios.

2.13 SQLAlchemy y acceso asíncrono

Un ORM (Object Relational Mapper) permite representar entidades de una base de datos mediante objetos del lenguaje de programación. SQLAlchemy es una herramienta de Python utilizada para mapear modelos, construir consultas y gestionar conexiones con bases de datos. El acceso asíncrono es especialmente útil en aplicaciones web donde varias operaciones pueden encontrarse en espera de recursos de E/S.

Gerardito utiliza SQLAlchemy en modalidad asíncrona junto con asyncpg. Esto permite integrar la persistencia con el backend FastAPI y mantener una estructura de acceso a datos separada de los endpoints. La combinación ayuda a mantener una arquitectura modular y facilita que futuras modificaciones del esquema no obliguen a reescribir la capa de presentación.

2.14 Integración continua y pruebas automatizadas

Las pruebas automatizadas permiten comprobar de manera repetible que el software conserva el comportamiento esperado después de modificaciones. Las pruebas unitarias verifican componentes individuales, mientras las pruebas de integración revisan la interacción entre partes del sistema. La integración continua (CI) automatiza la ejecución de estas comprobaciones ante eventos como push o pull request.

El proyecto utiliza pytest para la suite de pruebas y GitHub Actions para automatizar su ejecución. Se señala que el flujo establece un entorno Python, instala dependencias, establece conexión con Ollama Cloud y ejecuta la suite completa. De esta manera, se dispone de un mecanismo de control de calidad integrado en el repositorio.

2.15 Despliegue Continuo (Continuous Deployment)

El despliegue continuo, también conocido como Continuous Deployment, es una práctica de CI/CD que permite publicar automáticamente una nueva versión de una aplicación en un entorno de producción después de que los cambios han sido integrados al repositorio y cumplen con las condiciones definidas en el flujo de automatización.

En el proyecto Gerardito, el proceso de despliegue se encuentra automatizado a partir de la rama principal del repositorio. Cuando se realiza un push a la rama main, se activa un workflow configurado mediante GitHub Actions. Este workflow establece una conexión segura mediante SSH con la máquina virtual desplegada en Oracle Cloud, donde se encuentra alojada la aplicación.

Una vez establecida la conexión con el servidor, GitHub Actions ejecuta el script `deploy.sh`, encargado de actualizar la versión desplegada del sistema. El script obtiene el código más reciente mediante un `git pull` y posteriormente reconstruye y levanta los servicios definidos en Docker Compose, permitiendo que la nueva versión de Gerardito quede disponible en el entorno de producción sin necesidad de realizar manualmente cada uno de estos pasos.

El flujo puede representarse de la siguiente manera:

Push a main → GitHub Actions → conexión SSH → Oracle Cloud → `deploy.sh` → `git pull` → Docker Compose → nueva versión desplegada.

Este proceso aporta reproducibilidad y reduce los errores asociados con despliegues manuales, además de mantener una relación directa entre la versión almacenada en la rama principal del repositorio y la aplicación disponible públicamente.

2.16 Docker y Docker Compose

Los contenedores permiten empaquetar aplicaciones y dependencias en unidades reproducibles. Docker Compose agrega una capa de orquestación para definir y ejecutar varios servicios como una aplicación conjunta. En proyectos con frontend,

backend y base de datos, este enfoque ayuda a reducir la dependencia de configuraciones manuales en cada equipo.

Docker Compose define tres servicios: PostgreSQL, API y frontend. La contenerización permite que la aplicación pueda trasladarse a una máquina virtual sin instalar manualmente todos los componentes de la pila en el sistema operativo anfitrión.

2.17 Observabilidad y métricas

La observabilidad busca proporcionar información sobre el comportamiento interno de un sistema a partir de señales observables. En aplicaciones web resulta útil registrar tiempos de respuesta, errores y métricas relacionadas con los servicios externos. En una aplicación de IA, además, interesa distinguir el tiempo total de una solicitud del tiempo consumido por la inferencia del modelo.

Gerardito incorpora middleware para medir tiempos de respuesta, un decorador específico para registrar la inferencia del LLM y un endpoint /metrics con métricas acumuladas. Esta implementación ofrece una primera capa de observabilidad y permite identificar problemas de rendimiento.

2.18 Arquitectura desacoplada y separación por capas

Una arquitectura desacoplada procura disminuir la dependencia directa entre componentes. En una aplicación web moderna, la interfaz, la lógica de negocio, la inteligencia artificial y la persistencia pueden representarse como capas con responsabilidades distintas. La separación facilita el mantenimiento, las pruebas y la evolución tecnológica.

La arquitectura objetivo de Gerardito está organizada en cinco capas: interfaz, API/backend, servicio de inteligencia artificial, datos y operación. El frontend no necesita conocer la implementación interna de la IA y la API actúa como frontera de comunicación. Esta estructura representa una mejora respecto de etapas iniciales del proyecto y hace posible sustituir tecnologías específicas sin rehacer todo el sistema.

2.19 Ética y uso responsable en orientación vocacional

Los sistemas de recomendación aplicados a decisiones educativas deben considerarse herramientas de apoyo. Una recomendación generada por IA puede influir en la percepción que una persona tiene sobre sus capacidades y sobre las carreras disponibles, por lo que el resultado debe presentarse de manera orientativa y no como una decisión automática irrevocable.

El enfoque de recomendaciones se limita al catálogo de carreras de la UGB y entrega una explicación de afinidad. Además, el sistema registra reseñas para conocer la respuesta del usuario y no utiliza la recomendación como sustituto del proceso institucional. Esta decisión es coherente con el carácter de prototipo y con la necesidad de mantener supervisión humana sobre procesos de orientación.

2.20 Síntesis conceptual aplicada al proyecto

Los conceptos anteriores convergen en una arquitectura donde la interfaz captura lenguaje del usuario, la API controla el intercambio de información, los validadores delimitan la entrada y el servicio de IA interpreta el perfil con referencia al catálogo institucional. La persistencia conserva la información requerida para trazabilidad y la contenerización facilita que el conjunto sea reproducible. Esta integración representa la aplicación práctica de los conceptos estudiados durante la preespecialización.

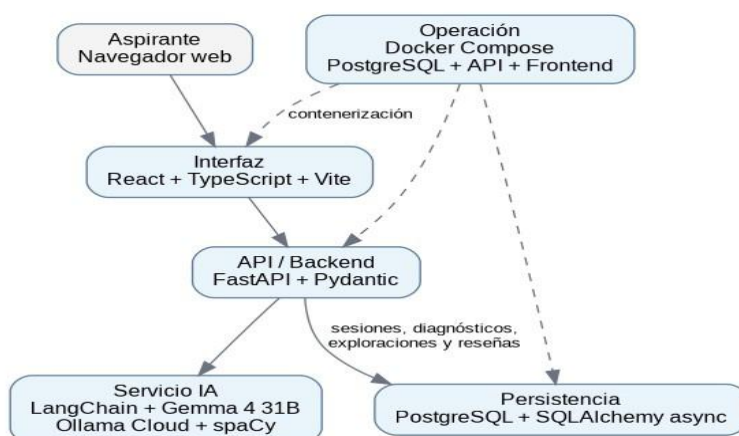


Figura 1. Arquitectura de cinco capas de Gerardito, elaborada a partir de la arquitectura actual documentada en el proyecto.

3. ANALISIS DEL PROBLEMA

3.1 Contexto de la situación problemática

La selección de una carrera universitaria requiere que el estudiante identifique intereses, habilidades, expectativas y que pueda relacionarlos con una oferta académica concreta. En el anteproyecto se planteó que los sistemas tradicionales de orientación pueden presentar limitaciones por su naturaleza estática, especialmente cuando dependen de formularios y reglas predefinidas.

La situación problemática abordada por Gerardito se centra en la distancia entre la manera flexible en que un aspirante expresa sus intereses y la forma rígida en que un sistema tradicional puede recibir y procesar esa información. Un estudiante puede escribir de manera informal, combinar varias ideas, utilizar expresiones locales o cometer errores ortográficos. Si el sistema solo acepta respuestas cerradas, parte de la información semántica potencialmente útil puede perderse.

3.2 Formulación del problema

¿Cómo desarrollar una aplicación inteligente que permita a los aspirantes expresar sus habilidades e intereses de forma flexible y que, a partir de esa información, genere recomendaciones de carreras universitarias pertenecientes al catálogo de la Universidad Gerardo Barrios, manteniendo validación de datos, controles de seguridad, persistencia de la interacción y una arquitectura web escalable?

3.3 Causas identificadas

- Interacción basada principalmente en formularios y opciones predeterminadas.
- Limitada capacidad de interpretar lenguaje natural y errores ortográficos.
- Dificultad para personalizar la explicación de las recomendaciones.
- Necesidad de integrar información de orientación con una oferta académica específica.
- Necesidad de disponer de una arquitectura que separe frontend, backend, IA y persistencia.

3.4 Consecuencias potenciales

- Recomendaciones percibidas como genéricas o poco cercanas al perfil del aspirante.
- Mayor fricción durante el proceso de orientación.
- Dificultad para explorar alternativas de manera conversacional.
- Menor trazabilidad de las interacciones si no existe persistencia.
- Necesidad de procesos adicionales para evaluar la satisfacción del usuario.

3.5 Usuarios y beneficiarios

La identificación de los usuarios y beneficiarios permite establecer a quiénes está dirigida la solución propuesta y de qué manera puede contribuir a sus necesidades. En este caso, se consideran tanto los aspirantes de bachillerato como la Universidad Gerardo Barrios, de acuerdo con los objetivos planteados para el proyecto.

A continuación, se presentan los principales usuarios y beneficiarios de la solución propuesta:

Usuario/Beneficiario	Necesidad principal	Beneficio de la aplicación
- Aspirantes de bachillerato.	Encontrar una carrera afín a sus gustos y habilidades sin fricciones tecnológicas.	Expresión libre y diagnóstico comprensible, justificado y validado en tiempo real
- Universidad Gerardo Barrios	Fortalecer los procesos de admisión y orientación.	Herramienta automatizada, escalable y alineada con el catálogo oficial.

3.6 Requerimientos funcionales identificados

1. Recibir habilidades e intereses mediante datos categóricos y texto libre.
2. Validar las solicitudes antes de enviarlas al servicio de IA.
3. Clasificar entradas válidas e inválidas mediante un filtro semántico.
4. Generar un diagnóstico usando el catálogo institucional.
5. Mostrar tres carreras sugeridas con porcentaje de compatibilidad y justificación.

6. Permitir explorar información adicional de una carrera.
7. Registrar una reseña final del usuario.
8. Analizar la reseña mediante técnicas de IA/NLP.
9. Persistir las sesiones y resultados en PostgreSQL.
10. Exponer la funcionalidad mediante una API documentada.

4. PROPUESTA DE SOLUCION

4.1 Descripción general

Gerardito es una aplicación web interactiva que automatiza y humaniza una parte del proceso de orientación vocacional. El sistema permite al usuario seleccionar y escribir habilidades e intereses, procesa esa información mediante una capa de seguridad y genera una matriz de afinidad con tres carreras sugeridas, su porcentaje de compatibilidad y una explicación. Seguidamente, el usuario puede explorar una de las opciones y dejar una reseña.

4.2 Objetivo general

Desarrollar e implementar un sistema web de orientación vocacional asistido por inteligencia artificial que interprete habilidades e intereses expresados por aspirantes y genere recomendaciones de carreras universitarias basadas en el catálogo institucional de la Universidad Gerardo Barrios, mediante una arquitectura desacoplada, validada, persistente y contenerizada.

4.3 Objetivos específicos

1. Desarrollar flujos de validación semántica orientados a la clasificación de perfiles vocacionales ingresados de forma libre por los estudiantes, aplicando tolerancia a fallos ortográficos.
2. Implementar una arquitectura segura que integre un LLM local y herramientas NLP en una interfaz web funcional.
3. Analizar la calidad de la interacción mediante un módulo de análisis de sentimientos híbrido para procesar la retroalimentación final de los usuarios.

4.3.1 Arquitectura de comunicación y API

El frontend se comunica con el backend mediante una API RESTful desarrollada con FastAPI y organizada bajo el prefijo /api/v1/. Los endpoints que manejan información de usuario requieren el encabezado X-Session-Id, un UUID generado por el frontend que permite relacionar de forma consistente la sesión con los registros almacenados en PostgreSQL.

- Validación de texto: POST /api/v1/validar-texto.
- Diagnóstico vocacional: POST /api/v1/diagnostico.
- Exploración de carrera: POST /api/v1/explorar.
- Evaluación de reseña: POST /api/v1/resena.
- Operación y diagnóstico: GET /api/v1/health, /metadata, /prueba-llm y /metrics.

4.4 Arquitectura de la solución

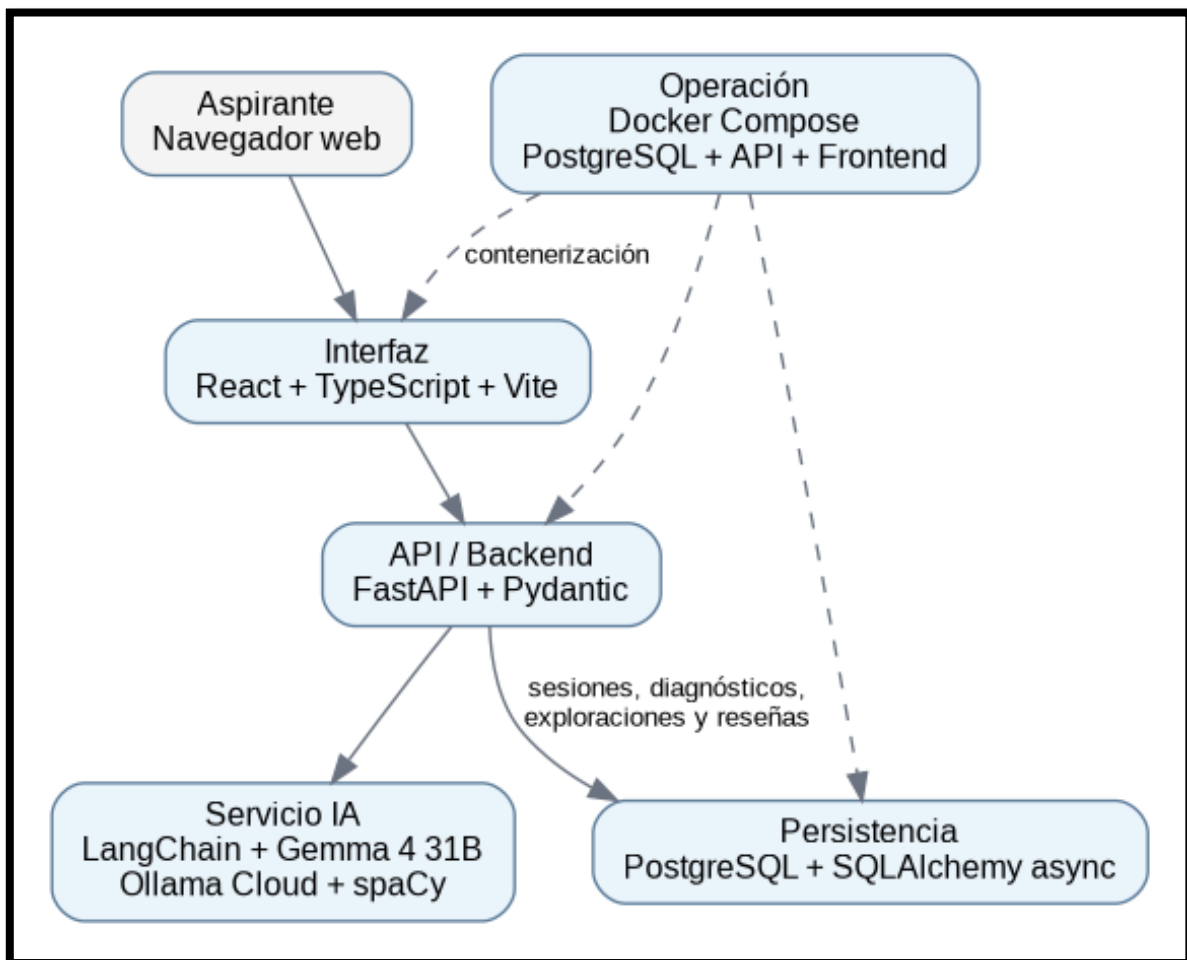


Figura 2. Arquitectura actual del sistema.

La arquitectura se organiza en cinco capas: interfaz, API/backend, servicio de IA, datos y operación. La interfaz se implementa con React + TypeScript + Vite; el backend se desarrolla con FastAPI y Pydantic; el servicio de inteligencia artificial utiliza Gemma 4 31B mediante Ollama Cloud y spaCy; la persistencia se resuelve con PostgreSQL y

SQLAlchemy en modalidad asíncrona; y la operación se gestiona mediante Docker Compose.

4.5 Flujo funcional de la aplicación

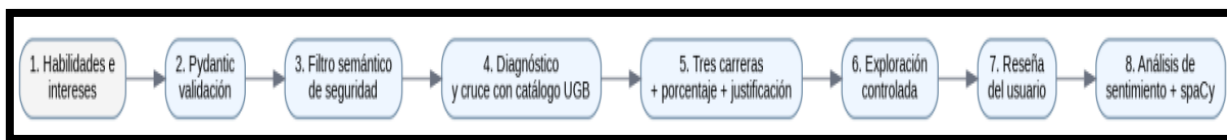


Figura 3. Flujo funcional de Gerardito.

El flujo inicia con el perfilamiento del aspirante. La información se valida y se envía al filtro semántico de seguridad. Cuando la entrada cumple las condiciones del sistema, se genera el diagnóstico y se realiza el cruce con el catálogo institucional.

La aplicación entrega tres recomendaciones, permite explorar una de ellas y finalmente solicita una reseña que pasa por el módulo de análisis de sentimiento. Este flujo mantiene separadas las funciones de captura, seguridad, recomendación y evaluación.

4.6 Componentes tecnológicos

Se integran diferentes componentes tecnológicos que permiten implementar y garantizar el funcionamiento de sus principales características. Cada tecnología cumple una función específica dentro de la arquitectura del sistema, contribuyendo a la interacción con el usuario, el procesamiento de la información y la ejecución de las funcionalidades establecidas.

Los componentes utilizados en el desarrollo son los siguientes:

Componente	Tecnología	Función
Frontend	React + TypeScript + Vite	Interfaz de usuario y wizard de cuatro fases.
Backend	FastAPI	API REST y orquestación de la lógica del sistema.

Validación	Pydantic	Validación y saneamiento de datos de entrada/salida.
IA generativa	Gemma 4 31B + Ollama Cloud	Diagnóstico, recomendación y análisis de lenguaje.
NLP	spaCy es_core_news_sm	Procesamiento lingüístico y apoyo al análisis de reseñas.
Persistencia	PostgreSQL	Almacenamiento de sesiones, diagnósticos, exploraciones y reseñas.
ORM	SQLAlchemy async + asyncpg	Acceso asíncrono a datos.
Contenedores	Docker + Docker Compose	Ejecución reproducible de los servicios.
Pruebas	pytest	Pruebas automatizadas.
CI/CD	GitHub Actions	Automatización de validación del repositorio.
Observabilidad	Middleware + /metrics	Medición de tiempos y métricas acumuladas.

4.7 Datos utilizados

Para el funcionamiento de la solución se utilizan diferentes tipos de datos, los cuales permiten procesar la información proporcionada por los aspirantes y generar recomendaciones de acuerdo con el catálogo académico institucional. Estos datos cumplen funciones específicas dentro del sistema, desde el perfilamiento del usuario hasta el almacenamiento de diagnósticos, exploraciones y reseñas.

Fuente	Tipo	Uso	Observaciones
Catálogo UGB	Texto estático	Cruce de variables de afinidad para recomendar carreras.	Integrado en los prompts del backend.

Sesión de usuario	Texto libre y categórico	Habilidades e intereses del aspirante.	Saneados mediante Pydantic.
Historial	PostgreSQL	Sesiones, diagnósticos, exploraciones y reseñas.	Persistente y preparado para concurrencia.

4.8 Infraestructura y despliegue

La versión actual del proyecto utiliza una máquina virtual en Oracle como infraestructura de ejecución y un dominio gratuito obtenido mediante DigitalPlat FreeDomain para facilitar el acceso al sistema.

4.9 Justificación de la solución

La solución elegida permite combinar una interfaz sencilla para el aspirante con un backend que centraliza validación, seguridad, persistencia e integración de inteligencia artificial. React facilita una interacción por etapas; FastAPI establece un contrato claro entre frontend y backend; PostgreSQL conserva los resultados; Docker simplifica la reproducción del entorno; y Gemma 4 31B vía Ollama Cloud permite utilizar un modelo de mayor escala sin trasladar el requisito de GPU a la máquina virtual. Esta última decisión introduce una dependencia de conectividad, pero reduce la carga de infraestructura local.

5. METODOLOGÍA DE DESARROLLO DEL PROYECTO

El desarrollo se organizó de forma incremental, incorporando componentes de la solución por etapas y validando la integración entre ellos. La metodología descrita a continuación se basa en el estado actual documentado en el repositorio del proyecto, la estructura del código y el plan por semanas incluido en el README.

5.1 Etapa 1. Análisis y definición

Se identificó como necesidad central mejorar la interacción del aspirante con el proceso de orientación y reducir la rigidez de una entrada exclusivamente estructurada. A partir del anteproyecto y de la evolución de los módulos se definieron las funcionalidades principales: captura de perfil, validación, diagnóstico, exploración y reseña.

5.2 Etapa 2. Diseño de arquitectura

Se adoptó una arquitectura desacoplada donde el frontend consume una API REST y el backend centraliza la lógica de negocio. Posteriormente se añadió PostgreSQL como capa persistente y Docker Compose como medio de orquestación. Esta separación permitió que la interfaz evolucionara desde prototipos anteriores hacia una SPA React sin trasladar la lógica de inferencia al cliente.

5.3 Etapa 3. Desarrollo del backend

FastAPI se utilizó para construir los endpoints del sistema. Pydantic se integró para describir y validar contratos de entrada y salida. La documentación automática mediante Swagger facilita la inspección y prueba de los servicios.

```
docker compose up --build  
  
python -m pytest -v test/
```

Figura 4. Comandos documentados para levantar el entorno y ejecutar pruebas automatizadas.

5.4 Etapa 4. Desarrollo del frontend

El frontend fue desarrollado como una SPA utilizando React, TypeScript y Vite. La interacción se dividió en cuatro fases, de modo que el usuario avance desde la captura de habilidades e intereses hacia el diagnóstico, la exploración y la reseña. La interfaz

consume únicamente los endpoints definidos por el backend, manteniendo el desacoplamiento de la lógica de negocio.

5.5 Etapa 5. Integración del modelo de IA

La capa de IA integra Gemma 4 31B mediante Ollama Cloud. La orquestación se realiza desde el backend y recibe como entrada información validada del perfil. La implementación incluye un filtro semántico para clasificar solicitudes y un flujo de diagnóstico que utiliza el catálogo de la UGB dentro del contexto. Ollama documenta que sus modelos cloud pueden ser accedidos programáticamente y que el acceso remoto requiere autenticación. [16]

5.6 Etapa 6. Procesamiento NLP

spaCy se utiliza con el modelo es_core_news_sm para apoyar el procesamiento de las reseñas. El enfoque híbrido permite separar el razonamiento generativo de las operaciones lingüísticas. El resultado final incluye una clasificación de sentimiento y procesamiento de palabras clave o estructura lingüística, según el flujo implementado en el backend.

5.7 Etapa 7. Persistencia

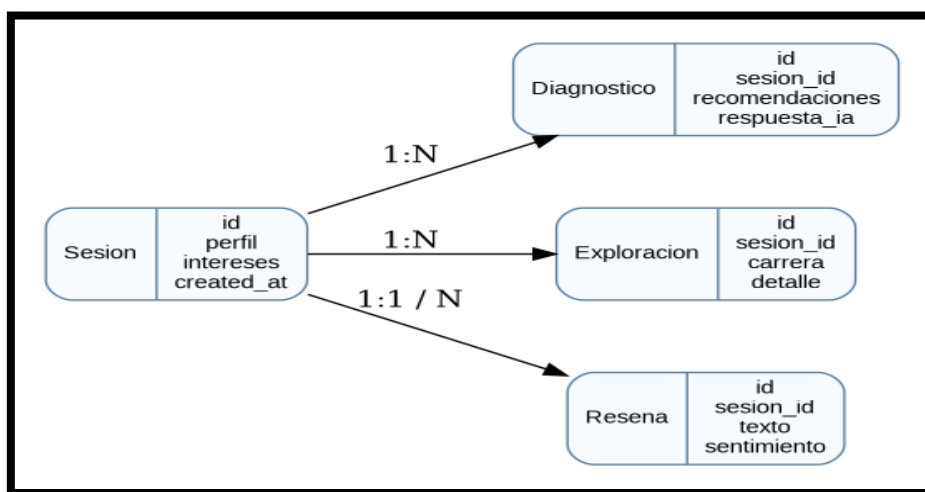


Figura 5. Modelo lógico simplificado de persistencia reportado por el proyecto.

La persistencia fue implementada con PostgreSQL y SQLAlchemy en modalidad asíncrona. Las estructuras principales corresponden a las entidades Sesión,

Diagnostico, Exploracion y Resena. El uso de una base de datos relacional elimina la dependencia de una memoria temporal y permite conservar el historial requerido por el proceso de orientación.

5.8 Etapa 8. Contenerización

El archivo docker-compose.yml define la orquestación de los servicios PostgreSQL, API y frontend. Esta estrategia facilita ejecutar el proyecto con un único comando y simplifica su traslado a una infraestructura distinta.

5.9 Etapa 9. Seguridad

La seguridad se abordó mediante validación estructural, filtros semánticos, restricciones del prompt y manejo controlado de credenciales. El texto libre no se procesa directamente por el LLM sin pasar por las capas previas. El riesgo de prompt injection se considera explícitamente dentro del diseño, aunque la mitigación implementada no debe interpretarse como protección absoluta. OWASP identifica la prompt injection como un riesgo relevante y recomienda un enfoque defensivo de múltiples capas. [12]

5.10 Etapa 10. Pruebas y CI/CD

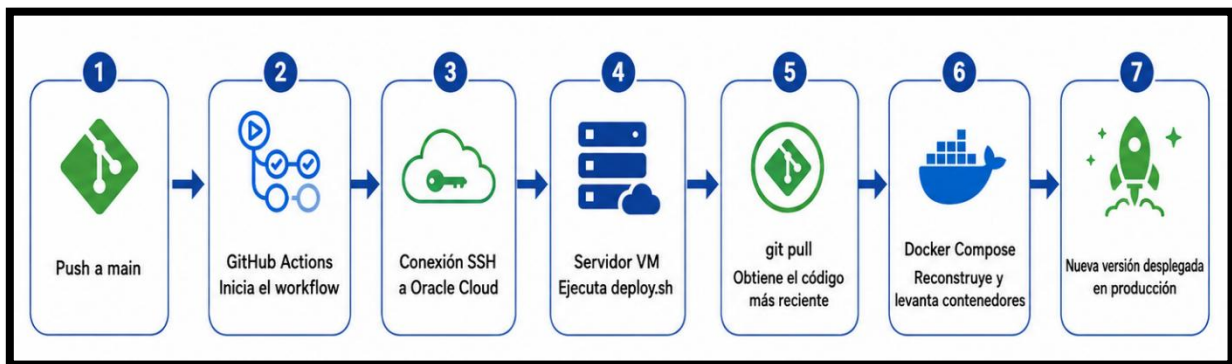
Las pruebas del sistema se automatizan mediante pytest y GitHub Actions. El flujo de integración continua se encarga de preparar el entorno de ejecución, instalar las dependencias requeridas, establecer conexión con Ollama Cloud y ejecutar la suite completa de pruebas ante cambios realizados en el repositorio.

Los endpoints que dependen directamente del modelo de lenguaje, como diagnóstico, exploración y evaluación de reseñas, requieren conectividad con Ollama Cloud para completar correctamente las pruebas automatizadas.

Además de la integración continua, el proyecto incorpora un proceso de despliegue automático. Cuando se realiza un push a la rama main, GitHub Actions activa el workflow de despliegue y establece una conexión segura mediante SSH con la máquina virtual alojada en Oracle Cloud.

Una vez establecida la conexión, se ejecuta el script `deploy.sh`, cuya función es obtener la versión más reciente del código mediante `git pull`, posteriormente reconstruir y levantar los servicios de la aplicación utilizando Docker Compose. De esta manera, la nueva versión del sistema queda desplegada automáticamente en el entorno de producción.

El flujo general puede resumirse de la siguiente forma:



Este mecanismo permite mantener una relación directa entre la rama principal del repositorio y la versión pública del sistema, reduciendo la intervención manual durante el proceso de actualización. Por otra parte, el procedimiento de rollback permanece documentado como un proceso manual para regresar a una versión estable en caso de fallos posteriores al despliegue.

5.11 Etapa 11. Observabilidad

Se añadió un middleware para registrar tiempos de respuesta, un mecanismo específico para la inferencia del LLM y un endpoint `/metrics` con información acumulada. Esta etapa permite conocer el comportamiento básico del sistema y constituye una base para implementar posteriormente monitoreo más completo.

5.12 Etapa 12. Despliegue

La implementación final incorpora una máquina virtual de Oracle y un dominio gratuito de DigitalPlat FreeDomain. En términos metodológicos, el despliegue representa la transición del entorno de desarrollo a un entorno accesible mediante red.

5.13 Etapa 13. Recuperación y rollback

Se documentó un procedimiento manual de rollback para escenarios en los que la URL pública deje de funcionar, aparezcan errores HTTP 500, falle la conexión con PostgreSQL o se presenten problemas con Ollama Cloud. El procedimiento parte de una versión estable, detiene los servicios, reconstruye el despliegue con Docker Compose y verifica los endpoints /health, /metadata y /validar-texto. La versión estable documentada es v1.0.0. La automatización del rollback mediante GitHub Actions se considera trabajo futuro.

5.14 Etapa 14. Verificación del despliegue

La verificación operativa se realiza mediante health checks y pruebas del flujo principal. La documentación de rollback registra como URL pública <https://gerarditoughb.qd.je> y utiliza los endpoints /api/v1/health y /api/v1/metadata para comprobar que el servicio esté disponible después de un despliegue o recuperación.

5.15 Integración final de las seis sesiones

La integración de las seis sesiones nos permite consolidar los aprendizajes y avances desarrollados durante el proyecto en una única versión coherente de Gerardito. Cada sesión aporta elementos que fortalecen la solución final, desde su arquitectura y funcionalidades hasta las pruebas, seguridad y despliegue.

Sesión	Aprendizaje integrado	Evidencia incorporada en Gerardito
1	Diagnóstico, README y arquitectura	Problema de orientación vocacional, usuarios, función de la IA, evolución desde el planteamiento inicial, arquitectura final, riesgos y deuda técnica.
2	API inteligente y contratos	API FastAPI bajo /api/v1/, contratos Pydantic, header

		XSession-Id, validaciones HTTP 400/422, manejo de errores, /health y /metadata.
3	Pruebas y CI/CD	Pruebas con pytest, workflow de GitHub Actions, variables 32 de entorno sin secretos y verificación automatizada en push/pull request.
4	Despliegue e infraestructura	Despliegue con Docker Compose en máquina virtual de Oracle, PostgreSQL, API y frontend; acceso público mediante https://gerarditoub.qd.je .
5	Observabilidad, rendimiento y escalabilidad	Middleware de tiempos, métricas persistidas en PostgreSQL, endpoint /metrics, análisis del cuello de botella del LLM y plan de escalabilidad.
6	Seguridad, release y rollback	Filtro semántico, validación de entradas, protección de secretos, release estable v1.0.0, commit trazable, pruebas de humo y procedimiento manual de rollback.

5.16 Contratos, salud, versión y trazabilidad

La API utiliza el prefijo /api/v1/. Los endpoints de datos emplean el encabezado X-Session-Id con un UUID para correlacionar la sesión del usuario. La versión puede consultarse mediante /api/v1/metadata y el estado del servicio mediante /api/v1/health. Esta separación permite demostrar operación, versión y trazabilidad sin exponer secretos.

Elemento	Implementación final
Prefijo API	/api/v1/
Sesión	X-Session-Id (UUID)
Salud	GET /api/v1/health
Metadatos	GET /api/v1/metadata
Métrica	GET /api/v1/metrics
Versión estable documentada	v1.0.0
Commit estable	4a4896a7e547b36e7374bfdf40439858ad09b211
URL pública	https://gerarditoughb.qd.je/

5.17 Release, pruebas de humo y rollback

Para continuidad operativa se documentó un rollback manual hacia la versión estable v1.0.0. El procedimiento contempla identificar el commit estable, detener los contenedores, reconstruir la versión seleccionada y ejecutar pruebas de humo sobre salud, metadatos, validación de texto y disponibilidad de PostgreSQL. El rollback automatizado mediante CI/CD se mantiene como mejora futura.

Prueba de humo 1: verificar /api/v1/health desde la URL pública.

Prueba de humo 2: verificar /api/v1/metadata y confirmar versión.

Prueba de humo 3: ejecutar una solicitud controlada a /api/v1/validar-texto.

Prueba de humo 4: comprobar disponibilidad de PostgreSQL con pg_isready.

Después del rollback se conservan logs del incidente y se comunica al equipo la versión restaurada.

6. RESULTADOS ALCANZADOS

6.1 Cumplimiento funcional

El cumplimiento funcional permite evidenciar el estado de las principales capacidades implementadas en Gerardito y su aporte al funcionamiento de la solución. A

continuación, se presentan los componentes desarrollados y el estado alcanzado en cada uno de ellos.

Área	Resultado alcanzado	Estado documentado
Backend	API REST FastAPI con 5 endpoints y Swagger.	Funcional
Persistencia	PostgreSQL + SQLAlchemy async; 4 tablas principales.	Funcional
Frontend	React + TypeScript + Vite; wizard de 4 fases.	Completo
IA	Gemma 4 31B vía Ollama Cloud + spaCy.	Operativo
Seguridad	Filtro semántico + validación Pydantic.	Funcional
Contenedores	Docker Compose con PostgreSQL, API y Frontend.	Completo
Observabilidad	Middleware + decorador LLM + /metrics.	Completo
CI/CD	GitHub Actions + pytest.	Completo

6.2 Resultado de la interfaz de usuario

La interfaz final se encuentra implementada como una SPA. El flujo se organiza en cuatro fases y conecta los componentes de captura, diagnóstico, exploración y reseña. El resultado esperado desde la perspectiva del usuario es una experiencia continua sin necesidad de recargar la aplicación entre etapas.

Gerardito - Orientación Vocacional

gerarditoug.b.qd.je

Gerardito
Sistema de Orientación Vocacional Inteligente - UGB

Paso 1 de 2 Paso 2 de 2

Pistas de tu Futuro

¿Cuáles son tus habilidades naturales?

- ☐ Soy el/la que siempre expone en los trabajos grupales y habla sin pena.
- ☐ Se me dan muy bien los números, las fórmulas matemáticas o la física.
- ☒ Soy bueno/a escuchando los problemas de mis amigos y dándoles consejos.
- ☐ Entiendo rápido cómo usar aplicaciones nuevas o arreglar cosas de la computadora.
- ☐ Dibujo muy bien, tengo buena letra o me fijo mucho en la estética y los colores.
- ☐ Tengo facilidad para redactar ensayos, leer mucho o aprender idiomas nuevos.
- ☐ Soy súper organizado/a, me gusta liderar el grupo y repartir las tareas.
- ☐ Soy hábil con las manos para armar maquetas, reparar cosas o construir.

Otra habilidad (Opcional):

Tengo facilidad para detectar errores ortográficos y de redacción

Siguiete Paso

6.3 Resultado del motor de recomendación

El resultado principal del motor consiste en tres carreras sugeridas con porcentaje de compatibilidad y una justificación. La recomendación se genera mediante la interpretación del perfil en texto libre y categórico, tomando como referencia el catálogo institucional. Este mecanismo se acerca más a una explicación semántica que a una predicción clásica de Machine Learning y refleja la arquitectura efectivamente implementada.

¡Hola! Qué perfil tan genial y versátil tienes. Me encanta que combines esa sensibilidad para ayudar a los demás con una mente emprendedora y un ojo creativo para el diseño; ¡esa es una mezcla ganadora para el mundo profesional actual!

Basado en tus habilidades y gustos, aquí tienes mis recomendaciones para ti en la UGB:

Carrera Sugerida	% de Afinidad	Por qué encaja
Arquitectura	95%	Se alinea perfectamente con tu interés por los planos, el diseño de casas, la decoración y los grandes edificios.
Licenciatura en Marketing y Negocios Digitales	90%	Combina tu pasión por crear contenido en redes sociales, el diseño gráfico y tu mentalidad para los negocios.
Licenciatura en Psicología	80%	Aprovecha tu habilidad natural para escuchar a los demás, dar consejos y tu capacidad de análisis humano.

Te invito a utilizar los botones para decidir tu futuro camino profesional.

¿Te gustaría profundizar en alguna de estas opciones?

☒ Si, profundizar en una carrera ☐ No, finalizar test

6.4 Resultado del filtro semántico

El filtro de seguridad está reportado como funcional y diseñado para tolerar errores ortográficos, al mismo tiempo que bloquea insultos, texto vacío o basura. Este componente reduce entradas que podrían producir respuestas fuera de contexto y funciona como una barrera previa al razonamiento del modelo.

Gerardito · Orientación Vocación x + Preguntar a Gemini

gerarditougbdje

Gmail YouTube Traducir Ver SS No Hay Para... Textos Todos los marcadores

Pistas de tu Futuro

¿Cuáles son tus habilidades naturales?

- ☒ Soy el/la que siempre expone en los trabajos grupales y habla sin pena.
- ☐ Se me dan muy bien los números, las fórmulas matemáticas o la física.
- ☐ Soy bueno/a escuchando los problemas de mis amigos y dándoles consejos.
- ☐ Entiendo rápido cómo usar aplicaciones nuevas o arreglar cosas de la computadora.
- ☐ Dibujo muy bien, tengo buena letra o me fijo mucho en la estética y los colores.
- ☒ Tengo facilidad para redactar ensayos, leer mucho o aprender idiomas nuevos.
- ☒ Soy súper organizado/a, me gusta liderar el grupo y repartir las tareas.
- ☐ Soy hábil con las manos para armar maquetas, reparar cosas o construir.

Otra habilidad (Opcional):

alkdiesieomd

El texto ingresado no es válido para una habilidad. Por favor, ingresa datos reales.

Siguiente Paso

6.5 Resultado de persistencia

La incorporación de PostgreSQL permite conservar las sesiones y resultados de la interacción. Esto resuelve la limitación de perder la información al cerrar o reiniciar la aplicación y abre la posibilidad de generar estadísticas institucionales en futuras versiones, siempre que se incorporen mecanismos de privacidad, autenticación y gobierno de datos.

```
[opc@instance-20260815-2223 gerardito]$ docker exec -it gerardito-postgres-1 psql -U gerardito -d gerardito_db
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

gerardito_db=# SELECT 'sesiones' as tabla, COUNT(*) FROM sesiones
UNION ALL SELECT 'diagnosticos', COUNT(*) FROM diagnosticos
UNION ALL SELECT 'exploraciones', COUNT(*) FROM exploraciones
UNION ALL SELECT 'resenas', COUNT(*) FROM resenas
UNION ALL SELECT 'metricas', COUNT(*) FROM metricas;
 tabla | count
-----+-----
 sesiones |      28
 diagnosticos |      28
 exploraciones |      20
 resenas |      15
 metricas |     255
(5 rows)
```

6.6 Resultado de pruebas y calidad

El proyecto dispone de una suite automatizada mediante pytest y un flujo de GitHub Actions. La implementación de pruebas automatizadas y CI/CD permite detectar posibles errores y regresiones durante el desarrollo, contribuyendo a mantener la estabilidad y calidad del sistema. La suite contempla también los endpoints que requieren inferencia mediante Ollama Cloud, por lo que su ejecución depende de la disponibilidad y conectividad con dicho servicio.

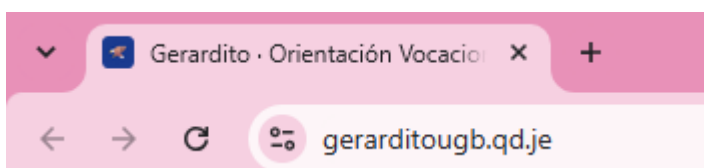
Run Title	Commit	Branch	Time
Merge pull request #8 from theb...	Deploy Gerardito #3: Commit a55a7d1	main	Today at 12:20 AM, 24s
Merge pull request #8 from theb...	Pruebas Gerardito API #35: Commit a55a7d1	main	Today at 12:20 AM, 1m 18s
fix/correcciones_finales	Pruebas Gerardito API #34: Pull request #8 opened by theboss0503	fix/correcciones_finales	Today at 12:19 AM, 1m 12s
Merge pull request #7 from the...	Deploy Gerardito #2: Commit 292f1b4	main	Aug 20, 8:57 PM CST, 1m 24s
Merge pull request #7 from the...	Pruebas Gerardito API #33: Commit 292f1b4	main	Aug 20, 8:57 PM CST, 1m 16s
Mejoras en produccion	Pruebas Gerardito API #32: Pull request #7 synchronize by theboss0503	mejoras_produccion	Aug 20, 8:56 PM CST, 1m 14s

6.7 Resultado de observabilidad

La aplicación dispone de métricas acumuladas y registro de tiempos de respuesta. Aunque esta funcionalidad no constituye una plataforma completa de observabilidad, sí proporciona indicadores iniciales para medir la operación y constituye un punto de partida para futuras mejoras de monitoreo y alertamiento.

6.8 Resultado de despliegue

Se incorporó una máquina virtual en Oracle y un dominio gratuito mediante DigitalPlat FreeDomain como parte de la entrega. Este resultado extiende el prototipo más allá del entorno local y facilita su demostración mediante acceso web.



6.9 Evidencias

Interfaz inicial de Gerardito desarrollada con React, TypeScript y Vite, donde el usuario inicia el proceso de orientación vocacional mediante un flujo guiado por etapas para ingresar sus habilidades e intereses.

A screenshot of the Gerardito web application interface. The page is titled 'Gerardito Sistema de Orientación Vocacional Inteligente - UGB'. It shows a guided flow for entering skills and interests. The current step is 'Paso 1 de 2'. The section is titled 'Pistas de tu Futuro' and asks '¿Cuáles son tus habilidades naturales?'. There are eight checkboxes with icons and text descriptions of skills. The first checkbox is checked: 'Soy bueno/a escuchando los problemas de mis amigos y dándoles consejos.' The other checkboxes are: 'Soy el/la que siempre expone en los trabajos grupales y habla sin pena.', 'Se me dan muy bien los números, las fórmulas matemáticas o la física.', 'Entiendo rápido cómo usar aplicaciones nuevas o arreglar cosas de la computadora.', 'Dibujó muy bien, tengo buena letra o me fijo mucho en la estética y los colores.', 'Tengo facilidad para redactar ensayos, leer mucho o aprender idiomas nuevos.', 'Soy súper organizado/a, me gusta liderar el grupo y repartir las tareas.', and 'Soy hábil con las manos para armar maquetas, reparar cosas o construir.' Below the checkboxes is a text input field for 'Otra habilidad (Opcional):' with the text 'Tengo facilidad para detectar errores ortográficos y de redacción.' A 'Siguiente Paso' button is at the bottom right.

Resultado del procesamiento de una solicitud válida, en el que Gerardito analiza el perfil del usuario y genera tres recomendaciones de carreras universitarias con su respectivo nivel de afinidad y justificación.

¡Hola! Qué perfil tan genial y versátil tienes. Me encanta que combines esa sensibilidad para ayudar a los demás con una mente emprendedora y un ojo creativo para el diseño; ¡esa es una mezcla ganadora para el mundo profesional actual!

Basado en tus habilidades y gustos, aquí tienes mis recomendaciones para ti en la UGB:

Carrera Sugerida	% de Afinidad	Por qué encaja
Arquitectura	95%	Se alinea perfectamente con tu interés por los planos, el diseño de casas, la decoración y los grandes edificios.
Licenciatura en Marketing y Negocios Digitales	90%	Combina tu pasión por crear contenido en redes sociales, el diseño gráfico y tu mentalidad para los negocios.
Licenciatura en Psicología	80%	Aprovecha tu habilidad natural para escuchar a los demás, dar consejos y tu capacidad de análisis humano.

Te invito a utilizar los botones para decidir tu futuro camino profesional.

¿Te gustaría profundizar en alguna de estas opciones?

☒ Si, profundizar en una carrera ☐ No, finalizar test

Vista de la fase de exploración, donde el usuario selecciona una de las carreras recomendadas y recibe información adicional generada por el sistema sobre dicha opción académica.

Arquitectura 95% Se alinea perfectamente con tu interés por los planos, el diseño de casas, la decoración y los grandes edificios.

Licenciatura en Marketing y Negocios Digitales 90% Combina tu pasión por crear contenido en redes sociales, el diseño gráfico y tu mentalidad para los negocios.

Licenciatura en Psicología 80% Aprovecha tu habilidad natural para escuchar a los demás, dar consejos y tu capacidad de análisis humano.

Te invito a utilizar los botones para decidir tu futuro camino profesional.

Selecciona la carrera que te gustaría explorar:

☐ Arquitectura

☐ Licenciatura en Marketing y Negocios Digitales

☒ Licenciatura en Psicología

Explorar Selección

Gerardito - Orientación Vocacional x + Preguntar a Gemini

gerarditoug.b.qd.je

Gmail YouTube Traducir Ver SS No Hay Para... Textos

Todos los marcadores

¡La Licenciatura en Psicología es una elección fascinante! Su principal ventaja es que te brinda las herramientas científicas para comprender el comportamiento humano, mejorar la salud mental y potenciar el bienestar emocional de las personas, permitiéndote generar un impacto positivo y real en la sociedad.

****Campo Laboral:****
****Psicología Clínica:**** Consulta privada, hospitales y centros de salud mental.
****Psicología Organizacional:**** Recursos Humanos, reclutamiento, capacitación y clima laboral en empresas.
****Psicología Educativa:**** Orientación escolar, apoyo psicopedagógico en colegios y universidades.
****Psicología Social:**** Trabajo en ONGs, proyectos comunitarios y políticas públicas.
****Investigación:**** Desarrollo de estudios sobre la mente y la conducta humana.

Para finalizar nuestra sesión, escribe una breve reseña evaluando mi servicio:

Tu opinión nos ayuda a mejorar...

Enviar Reseña

Registro de la retroalimentación proporcionada por el usuario y resultado del análisis de sentimiento realizado mediante un enfoque híbrido con Gemma 4 y spaCy, clasificando la opinión como positiva, negativa, neutral o inválida.

Para finalizar nuestra sesión, escribe una breve reseña evaluando mi servicio:

La experiencia con Gerardito fue muy buena. Las preguntas fueron claras y fáciles de responder, y las recomendaciones que recibí fueron interesantes.

Enviar Reseña

 **Gerardito**
 Sistema de Orientación Vocacional Inteligente · UGB

Sesión Finalizada 🎉

La sesión ha finalizado exitosamente. Tus respuestas han sido procesadas.

Sentimiento detectado: 😊 Positivo

Palabras clave: experiencia, buena, preguntas, claras, fáciles, recomendaciones, interesantes

Volver a hacer el test

Documentación interactiva de la API REST desarrollada con FastAPI, donde se visualizan los endpoints disponibles para validación, diagnóstico, exploración, reseñas, salud, metadatos y métricas.

Gerardito IA API 1.0.0 OAS 3.1
 /openapi.json
 API RESTful para motor de emparejamiento vocacional híbrido

Sistema		^
GET	/api/v1/health Health Check	▼
GET	/api/v1/metadata Get Metadata	▼
GET	/api/v1/prueba-11m Prueba Lim	▼
Fase 1: Recoleccion		^
POST	/api/v1/validar-texto Validar Texto	▼
Fase 2: Afinidad		^
POST	/api/v1/diagnostico Diagnostico	▼
Fase 3: Exploracion		^
POST	/api/v1/explorar Explorar	▼
Fase 4: Feedback		^
POST	/api/v1/resena Analizar Resena	▼
Observabilidad		^
GET	/api/v1/metrics Get Metrics	▼
default		^
GET	/ Read Root	▼

Evidencia del despliegue de Gerardito en la máquina virtual de Oracle Cloud, mostrando la ejecución y estado de los servicios contenerizados mediante Docker Compose.

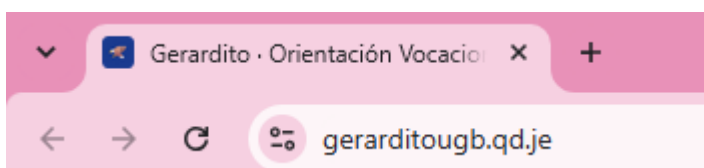
```
[opc@instance-20260815-2223 ~]$ docker ps
CONTAINER ID   IMAGE                     COMMAND                  CREATED        STATUS        PORTS
415dc3c3debc   gerardito-api             "uvicorn app.main:ap..." 21 hours ago   Up 21 hours (healthy)  8000/tcp
b48992fce7af   gerardito-frontend        "caddy file-server -..." 21 hours ago   Up 21 hours      80/tcp, 443/tcp, 2019/tcp, 5173/tcp, 443/udp
387cae709c56   caddy:2-alpine            "caddy run --config ..." 2 days ago     Up 2 days      0.0.0.0:80->80/tcp, [::]:80->80/tcp, 0.0.0.0:4
43->443/tcp, [::]:443->443/tcp, 443/udp, 2019/tcp
8bf7768d025f   postgres:16               "docker-entrypoint.s..." 2 days ago     Up 2 days (healthy)  5432/tcp
gerardito-postgres-1
```

Evidencia de la persistencia de información en PostgreSQL, donde se almacenan datos relacionados con sesiones, diagnósticos, exploraciones, reseñas y métricas generadas por el sistema.

```
[opc@instance-20260815-2223 gerardito]$ docker exec -it gerardito-postgres-1 psql -U gerardito -d gerardito_db
psql (16.15 (Debian 16.15-1.pgdg13+2))
Type "help" for help.

gerardito_db=# SELECT 'sesiones' as tabla, COUNT(*) FROM sesiones
UNION ALL SELECT 'diagnosticos', COUNT(*) FROM diagnosticos
UNION ALL SELECT 'exploraciones', COUNT(*) FROM exploraciones
UNION ALL SELECT 'resenas', COUNT(*) FROM resenas
UNION ALL SELECT 'metricas', COUNT(*) FROM metricas;
 tabla | count
-----+-----
 sesiones |      28
 diagnosticos |      28
 exploraciones |      20
 resenas |      15
 metricas |     255
(5 rows)
```

Evidencia de la disponibilidad pública de Gerardito mediante su dominio web, demostrando que la aplicación puede ser accedida desde Internet fuera del entorno local de desarrollo.



6.10 Robustez operacional

Como resultado adicional, Gerardito dispone de mecanismos básicos para operar y diagnosticar el sistema: health check, metadatos, prueba de conectividad con el LLM, métricas persistentes y un procedimiento de rollback. Esto permite detectar fallos, medir tiempos de respuesta y restaurar una versión estable sin reconstruir manualmente cada componente por separado.

6.11 Evidencia de rendimiento y observabilidad

El endpoint `/api/v1/metrics` consolida tiempos de respuesta, tiempo de inferencia LLM, tasa de error y últimas peticiones. La documentación técnica presenta valores de referencia del sistema; estos deben interpretarse como evidencia del release medido y no como una garantía permanente de rendimiento.

Indicador documentado	Valor / interpretación
Tiempo máximo global	11,771 ms
Tasa de error global	18.12%
Diagnóstico	28 llamadas; máximo 11,771 ms; 1 error
Explorar	12 llamadas; máximo 6,809 ms; 0 errores
Reseña	13 llamadas; máximo 6,418 ms; 3 errores
Validar texto	13 llamadas; máximo 7,991 ms; 0 errores
Cuello de botella principal	Inferencia del LLM / dependencia de Ollama Cloud

7. CONCLUSIONES GENERALES

7.1 Cumplimiento del objetivo del proyecto

Se logró desarrollar un prototipo funcional de orientación vocacional que integra una interfaz web moderna, una API REST, persistencia relacional, procesamiento de lenguaje natural y un modelo de lenguaje generativo. El sistema permite que el aspirante exprese habilidades e intereses y reciba recomendaciones de carreras pertenecientes al catálogo institucional de la Universidad Gerardo Barrios.

7.2 Aporte tecnológico

El principal aporte tecnológico consiste en demostrar la integración de diferentes capas de una aplicación de IA en un solo producto: React para la interfaz, FastAPI para el backend, Pydantic para validación, Gemma 4 31B mediante Ollama Cloud para la IA, spaCy para NLP, PostgreSQL para persistencia y Docker Compose para operación. Esta combinación brinda una arquitectura organizada y reutilizable para futuros proyectos de inteligencia artificial.

7.3 Aporte a la comunidad educativa

Gerardito aporta una alternativa tecnológica para apoyar a aspirantes que necesitan explorar opciones académicas de manera más flexible. La posibilidad de expresarse en lenguaje libre reduce la rigidez de los formularios y permite que la interacción sea más cercana al lenguaje cotidiano. El sistema no sustituye la orientación profesional, pero puede funcionar como mecanismo inicial de exploración y preparación para una decisión informada.

7.4 Nuevos conocimientos adquiridos

Durante el desarrollo se consolidaron conocimientos en integración de LLM, ingeniería de prompts, procesamiento de lenguaje natural, desarrollo de APIs, frontend con React y TypeScript, persistencia con PostgreSQL, ORM, contenerización, pruebas automatizadas, CI/CD, observabilidad y despliegue en infraestructura virtual. También se comprendió que una aplicación de IA requiere controles adicionales de seguridad,

validación y observabilidad que no son necesarios en la misma forma en una aplicación web convencional.

7.5 Limitaciones actuales

- Dependencia de la conectividad con Ollama Cloud para la inferencia del modelo.
- Ausencia de autenticación de usuarios; el `session_id` se genera como UUID.
- Necesidad de incorporar Alembic para una estrategia de migraciones de esquema más robusta.
- El sistema requiere evidencia experimental más amplia para cuantificar calidad de las recomendaciones mediante métricas formales comparables.
- La infraestructura pública depende de una máquina virtual en Oracle y de resolución DNS externa. La URL pública documentada para el release es <https://gerarditoughb.qd.je>; la continuidad del servicio depende de la disponibilidad de la VM, DNS, SSL y proveedor de IA.

7.6 Escalabilidad y trabajo futuro

El plan de crecimiento de Gerardito contempla, a corto plazo, caché con Redis, rate limiting y optimización del pool de conexiones. A mediano plazo se propone incorporar workers asíncronos para las llamadas al LLM, réplicas de lectura de PostgreSQL y CDN para el frontend. A largo plazo se plantea una evolución hacia microservicios y auto-scaling en Oracle Cloud. Estas mejoras responden al principal cuello de botella identificado: la latencia de inferencia del modelo externo y la necesidad de preparar el sistema para una mayor concurrencia.

Las limitaciones operativas que permanecen son la dependencia de Ollama Cloud, ausencia de autenticación de usuarios, falta de backup automatizado de PostgreSQL, ausencia de rate limiting explícito, rollback manual y falta de percentiles p50/p95 de latencia LLM en tiempo real. Estas limitaciones no impiden la demostración funcional, pero deben atenderse antes de un uso institucional intensivo.

7.7 Riesgo prioritario, límites y continuidad

El riesgo operativo prioritario es la dependencia del proveedor externo de inferencia. Una caída, latencia elevada o agotamiento de cuota de Ollama Cloud puede afectar diagnóstico, exploración y análisis de reseñas. Los controles actuales son validación de entradas, manejo de errores, métricas, health checks, variables de entorno para secretos y rollback manual. Permanecen como límites la ausencia de rate limiting, backups automáticos y rollback automático.

8. REFERENCIAS ESTUDIADAS

Docker. (s. f.). *Docker Compose documentation*. <https://docs.docker.com/compose/>

FastAPI. (2026). *Features and documentation*. <https://fastapi.tiangolo.com/features/>

GitHub. (s. f.). *GitHub Actions documentation*. <https://docs.github.com/actions>

Google DeepMind. (2026). *Gemma 4 model card*. Google AI for Developers. https://ai.google.dev/gemma/docs/core/model_card_4

Instituto Superior Universitario Rumiñahui. (2021). *Integrando tecnologías asociadas a data science en procesos de orientación vocacional y profesional*. <https://revista.ister.edu.ec/ojs/index.php/ISTER/article/view/20>

OWASP Foundation. (2025). *OWASP Top 10 for Large Language Model Applications, v2.0*. <https://owasp.org/www-project-top-10-for-large-language-model-applications/>

Pérez-Díaz, M. (2024). Aplicación de inteligencia artificial en la educación de América Latina: Tendencias, beneficios y desafíos. *Revista Veritas*. <https://revistaveritas.org/index.php/veritas/article/view/52>

PostgreSQL Global Development Group. (s. f.). *PostgreSQL documentation*. <https://www.postgresql.org/docs/>

React. (2026). *Using TypeScript. React Documentation*. <https://react.dev/learn/typescript>

spaCy. (s. f.). *Industrial-strength natural language processing in Python*.
<https://spacy.io/>

Universidad ORT Uruguay. (2025). Inteligencia artificial en educación superior: Revisión integrativa de la literatura. *Cuadernos de Investigación Educativa*.
<https://revistas.ort.edu.uy/cuadernos-de-investigacion-educativa/article/view/4034>

Universidad Técnica de Machala. (2024). *Desarrollo de una aplicación de orientación vocacional para bachilleres mediante test utilizando inteligencia artificial para la toma de decisiones*. Repositorio Institucional de la Universidad Técnica de Machala.
<https://repositorio.utmachala.edu.ec/items/9c22ef84-fd35-44c3-ad6ce7bd3bb66105>